Projekt zaliczeniowy

Przedstawia: Anna Michalak

Numer albumu: 59740

Data: 17.06.2026 r

GitHub: *https://github.com/AnnaMichalak2002/projekt*

System rezerwacji sal konferencyjnych

Cel systemu:

Celem systemu jest umożliwienie pracownikom organizacji rezerwowania sal konferencyjnych na określony przedział czasu w sposób uporządkowany, bezpieczny i zgodny z obowiązującymi regułami biznesowymi. System kontroluje dostępność sal, poprawność danych wejściowych, konflikty terminów oraz uprawnienia użytkowników wykonujących operacje.

Opis systemu:

Projekt przedstawia system rezerwacji sal konferencyjnych przeznaczony dla pracowników organizacji. System umożliwia zalogowanym użytkownikom przeglądanie dostępnych sal, tworzenie rezerwacji, przeglądanie własnych rezerwacji oraz anulowanie rezerwacji zgodnie z przyjętymi zasadami. Użytkownik z rolą administratora posiada dodatkowo możliwość zarządzania salami konferencyjnymi oraz przeglądania wszystkich rezerwacji w systemie.
W aplikacji zaimplementowano mechanizmy uwierzytelniania, autoryzacji, ochrony integralności wybranych żądań, walidacji danych wejściowych oraz obsługi błędów. Dodatkowo system wysyła użytkownikowi wiadomość e-mail potwierdzającą poprawne utworzenie rezerwacji.

Aktorzy w systemie:

1. Pracownik (USER) - może:

- przeglądać dostępne sale,
- tworzyć rezerwacje,
- przeglądać własne rezerwacje,
- anulować własne rezerwacje.

2. Administrator (ADMIN) - może:

- robić wszystko to, co USER,
- dodawać nowe sale,
- edytować dane sal,
- zmieniać status sali,
- przeglądać wszystkie rezerwacje,
- anulować dowolną rezerwację.

Zrealizowana funkcjonalność biznesowa

System realizuje funkcjonalność rezerwacji sal konferencyjnych w organizacji. Głównym celem biznesowym aplikacji jest umożliwienie pracownikom planowania spotkań w sposób uporządkowany i eliminujący konflikty terminów.

W ramach systemu użytkownik może przeglądać dostępne sale, sprawdzać ich szczegóły, tworzyć rezerwacje na określony przedział czasu, przeglądać własne rezerwacje oraz anulować własne rezerwacje, o ile spełnione są warunki biznesowe.

Administrator, oprócz uprawnień zwykłego użytkownika, może także zarządzać danymi sal konferencyjnych, zmieniać ich status oraz przeglądać wszystkie rezerwacje utworzone w systemie.

System wspiera podstawowe reguły organizacyjne związane z rezerwacją zasobów, takie jak:

- brak możliwości rezerwacji sali niedostępnej,
- brak możliwości utworzenia dwóch nakładających się rezerwacji tej samej sali,
- brak możliwości utworzenia dwóch aktywnych rezerwacji tego samego użytkownika w tym samym czasie,
- ograniczenie rezerwacji do dni roboczych i ustalonych godzin pracy organizacji,
- kontrola poprawności przedziału czasowego i czasu trwania spotkania.

Dodatkowo po poprawnym utworzeniu rezerwacji system generuje powiadomienie e-mail, co zwiększa użyteczność aplikacji i potwierdza wykonanie operacji.

Architektura systemu:

System został zaprojektowany w architekturze warstwowej, charakterystycznej dla aplikacji backendowych opartych na frameworku Spring Boot. Poszczególne elementy systemu zostały rozdzielone zgodnie z odpowiedzialnością, co zwiększa czytelność kodu, ułatwia testowanie oraz rozwój aplikacji.

W architekturze systemu można wyróżnić następujące warstwy:

1. Warstwa API / kontrolerów
Warstwa ta odpowiada za obsługę żądań HTTP oraz komunikację z klientem API. Zawiera klasy kontrolerów implementujące wygenerowane interfejsy OpenAPI, takie jak UsersController, RoomsController oraz ReservationsController. Kontrolery odbierają żądania, przekazują dane do warstwy serwisowej oraz zwracają odpowiedzi zgodne ze specyfikacją API.

2. Warstwa serwisowa
Warstwa serwisowa realizuje logikę biznesową systemu. Odpowiada za weryfikację reguł biznesowych, walidację operacji, sprawdzanie konfliktów rezerwacji, kontrolę dostępności sal oraz wywoływanie dodatkowych usług, takich jak wysyłka powiadomień e-mail. W tej warstwie znajdują się m.in. ReservationService, CurrentUserService, HmacService oraz NotificationService.

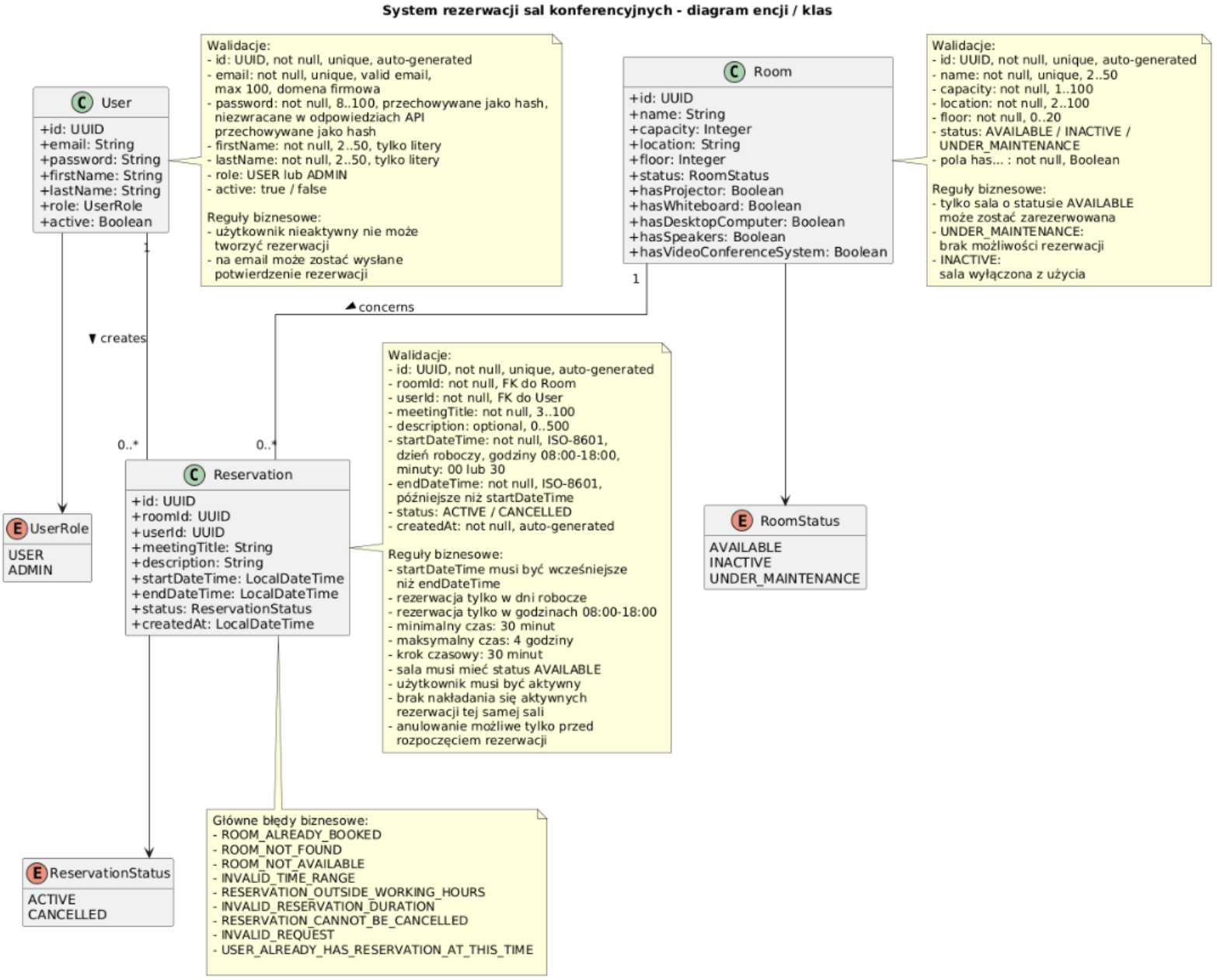
3. Warstwa przechowywania danych
Warstwa ta odpowiada za przechowywanie i udostępnianie danych aplikacji. W projekcie wykorzystano klasy typu store, np. RoomStore i ReservationStore, które przechowują dane o salach i rezerwacjach. Rozwiązanie to jest wystarczające dla projektu demonstracyjnego i upraszcza implementację.

4. Warstwa bezpieczeństwa
Za bezpieczeństwo systemu odpowiada konfiguracja Spring Security. Warstwa ta realizuje uwierzytelnianie użytkowników, kontrolę dostępu do zasobów oraz obsługę sytuacji związanych z brakiem autoryzacji lub uwierzytelnienia.

5. Warstwa integracyjna
System komunikuje się z zewnętrznym serwerem SMTP w celu wysyłania wiadomości e-mail potwierdzających utworzenie rezerwacji.

Przyjęta architektura warstwowa została uznana za odpowiednią dla projektowanego systemu, ponieważ pozwala oddzielić logikę biznesową od warstwy komunikacyjnej i bezpieczeństwa, a także zwiększa przejrzystość implementacji.

Diagram klas / encji



Opisane encje:

1. User

Reprezentuje pracownika korzystającego z systemu rezerwacji sal konferencyjnych.

id

Unikalny identyfikator użytkownika w systemie.

- typ danych: UUID
- znaczenie: jednoznacznie identyfikuje użytkownika
- walidacje:
    - not null
    - unique
    - generowane automatycznie
- przykładowa wartość:
    - 550e8400-e29b-41d4-a716-446655440000

email

Służbowy adres e-mail użytkownika, wykorzystywany jednocześnie jako login do systemu.

- typ danych: String
- znaczenie: identyfikuje użytkownika w systemie, służy do logowania oraz do wysyłania potwierdzeń rezerwacji przez system
- walidacje:
    - not null
    - unique
    - poprawny format e-mail
    - długość: maksymalnie 100 znaków
    - musi należeć do domeny firmowej @yourfurniture.pl
- przykładowa wartość:
    - jan.kowalski@yourfurniture.pl

password

Hasło użytkownika służące do uwierzytelniania.

- typ danych: String
- znaczenie: tajne hasło użytkownika wykorzystywane przy logowaniu
- walidacje:
    - not null
    - długość: minimum 8, maksimum 100 znaków

    - hasło jest przechowywane w postaci zahashowanej i nie jest zwracane w odpowiedziach API
- przykładowa wartość:
    - JanKowalski123!

firstName

Imię użytkownika.

- typ danych: String
- znaczenie: imię pracownika
- walidacje:
    - not null
    - długość: 2–50 znaków
    - tylko litery
- przykładowa wartość:
    - Jan

lastName

Nazwisko użytkownika.

- typ danych: String
- znaczenie: nazwisko pracownika
- walidacje:
    - not null
    - długość: 2–50 znaków
    - tylko litery
- przykładowa wartość:
    - Kowalski

role

Rola użytkownika w systemie.

- typ danych: Enum
- znaczenie: określa poziom uprawnień użytkownika
- walidacje:
    - not null
    - dozwolone wartości:
        - USER
        - ADMIN
- przykładowa wartość:
    - USER

active

Informacja, czy konto użytkownika jest aktywne.

- typ danych: Boolean
- znaczenie: określa, czy użytkownik może korzystać z systemu
- walidacje:
    - not null
    - dozwolone wartości:
        - true
        - false
- przykładowa wartość:
    - true

Reguły biznesowe

- użytkownik nieaktywny nie może tworzyć rezerwacji
- na adres e-mail użytkownika może być wysyłane potwierdzenie poprawnego utworzenia rezerwacji

2. Room

Reprezentuje salę konferencyjną dostępną w budynku organizacji.

Pola:

id

Unikalny identyfikator sali konferencyjnej.

- typ danych: UUID
- znaczenie: jednoznacznie identyfikuje salę w systemie
- walidacje:
    - not null
    - unique
    - generowane automatycznie
- przykładowa wartość:
    - 660e8400-e29b-41d4-a716-446655440001

name

Nazwa sali konferencyjnej.

- typ danych: String
- znaczenie: nazwa używana do identyfikacji sali przez użytkowników
- walidacje:
    - not null
    - unique
    - długość: 2–50 znaków

- przykładowa wartość:
    - Sala Konferencyjna A

capacity

Maksymalna liczba osób, które mogą przebywać w sali.

- typ danych: Integer
- znaczenie: określa pojemność sali konferencyjnej
- walidacje:
    - not null
    - minimum 1
    - maksimum 100
- przykładowa wartość:
    - 12

location

Lokalizacja sali w budynku.

- typ danych: String
- znaczenie: opisuje, gdzie znajduje się sala
- walidacje:
    - not null
    - długość: 2–100 znaków
- przykładowa wartość:
    - Budynek A, skrzydło wschodnie

floor

Piętro, na którym znajduje się sala.

- typ danych: Integer
- znaczenie: numer piętra w budynku
- walidacje:
    - not null
    - minimum 0
    - maksimum 20
- przykładowa wartość:
    - 2

status

Aktualny status dostępności sali.

- typ danych: Enum
- znaczenie: określa, czy sala może być rezerwowana

- walidacje:
    - not null
    - dozwolone wartości:
        - AVAILABLE
        - INACTIVE
        - UNDER_MAINTENANCE
- przykładowa wartość:
    - AVAILABLE

hasProjector

Informacja, czy sala posiada projektor.

- typ danych: Boolean
- znaczenie: określa dostępność projektora
- walidacje:
    - not null
- przykładowa wartość:
    - true

hasWhiteboard

Informacja, czy sala posiada tablicę.

- typ danych: Boolean
- znaczenie: określa dostępność tablicy
- walidacje:
    - not null
- przykładowa wartość:
    - true

hasDesktopComputer

Informacja, czy sala posiada komputer stacjonarny.

- typ danych: Boolean
- znaczenie: określa dostępność komputera stacjonarnego
- walidacje:
    - not null
- przykładowa wartość:
    - false

hasSpeakers

Informacja, czy sala posiada głośniki.

- typ danych: Boolean

- znaczenie: określa dostępność systemu audio
- walidacje:
    - not null
- przykładowa wartość:
    - true

hasVideoConferenceSystem

Informacja, czy sala umożliwia prowadzenie wideokonferencji.

- typ danych: Boolean
- znaczenie: określa dostępność systemu wideokonferencyjnego
- walidacje:
    - not null
- przykładowa wartość:
    - false

Reguły biznesowe

- tylko sala o statusie AVAILABLE może zostać zarezerwowana
- sala o statusie UNDER_MAINTENANCE nie może być rezerwowana
- sala o statusie INACTIVE jest wyłączona z użytkowania

3. Reservation

Reprezentuje rezerwację sali konferencyjnej dokonaną przez użytkownika.

Pola:

id

Unikalny identyfikator rezerwacji.

- typ danych: UUID
- znaczenie: jednoznacznie identyfikuje rezerwację w systemie
- walidacje:
    - not null
    - unique
    - generowane automatycznie
- przykładowa wartość:
    - 770e8400-e29b-41d4-a716-446655440002

roomId

Identyfikator sali, której dotyczy rezerwacja.

- typ danych: UUID
- znaczenie: wskazuje salę konferencyjną przypisaną do rezerwacji
- walidacje:
    - not null
    - klucz obcy do Room
    - musi wskazywać istniejącą salę
- przykładowa wartość:
    - 660e8400-e29b-41d4-a716-446655440001

userId

Identyfikator użytkownika, który utworzył rezerwację.

- typ danych: UUID
- znaczenie: wskazuje pracownika odpowiedzialnego za rezerwację
- walidacje:
    - not null
    - klucz obcy do User
    - musi wskazywać istniejącego użytkownika
- przykładowa wartość:
    - 550e8400-e29b-41d4-a716-446655440000

meetingTitle

Tytuł lub temat spotkania.

- typ danych: String
- znaczenie: krótki opis celu rezerwacji
- walidacje:
    - not null
    - długość: 3–100 znaków
- przykładowa wartość:
    - Analiza ofert i wybór importera drewna

description

Dodatkowy opis spotkania.

- typ danych: String
- znaczenie: bardziej szczegółowy opis rezerwacji lub celu spotkania
- walidacje:
    - opcjonalne
    - długość: 0–500 znaków
- przykładowa wartość:

    - Optymalizacja łańcucha dostaw: Wybór importera surowca drzewnego

startDateTime

Data i godzina rozpoczęcia rezerwacji.

- typ danych: LocalDateTime
- znaczenie: określa moment, od którego sala konferencyjna zostaje zarezerwowana
- walidacje:
    - not null
    - pełna data i godzina
    - musi wskazywać termin przyszły lub co najmniej aktualny
    - może przypadać wyłącznie na dzień roboczy, od poniedziałku do piątku
    - godzina rozpoczęcia musi mieścić się w godzinach pracy organizacji, tj. między 08:00 a 18:00
    - minuta rozpoczęcia może przyjmować wyłącznie wartości 00 albo 30, aby rezerwacje były tworzone z dokładnością do 30 minut
    - format ISO-8601
- przykładowa wartość:
    - 2026-06-15T10:00:00

endDateTime

Data i godzina zakończenia rezerwacji.

- typ danych: LocalDateTime
- znaczenie: określa moment, do którego sala pozostaje zarezerwowana
- walidacje:
    - not null
    - pełna data i godzina
    - musi być późniejsza niż startDateTime
    - format ISO-8601
- przykładowa wartość:
    - 2026-06-15T11:30:00

status

Status rezerwacji.

- typ danych: Enum
- znaczenie: określa aktualny stan rezerwacji
- walidacje:
    - not null
    - dozwolone wartości:

        - ACTIVE
        - CANCELLED
- przykładowa wartość:
    - ACTIVE

createdAt

Data i godzina utworzenia rezerwacji.

- typ danych: LocalDateTime
- znaczenie: moment zapisania rezerwacji w systemie
- walidacje:
    - not null
    - ustawiane automatycznie przy utworzeniu rezerwacji
- przykładowa wartość:
    - 2026-05-20T14:32:10

Reguły biznesowe:

- data i godzina rozpoczęcia rezerwacji musi być wcześniejsza niż data i godzina jej zakończenia
- rezerwacja może zostać utworzona wyłącznie w dni robocze, od poniedziałku do piątku
- rezerwacja może odbywać się tylko w godzinach pracy organizacji, tj. od 08:00 do 18:00
- minimalny czas trwania jednej rezerwacji wynosi 30 minut
- maksymalny czas trwania jednej rezerwacji wynosi 4 godziny
- godzina rozpoczęcia i zakończenia rezerwacji musi być ustawiona z dokładnością do 30 minut
- rezerwacja nie może zostać utworzona dla sali o statusie innym niż AVAILABLE
- użytkownik tworzący rezerwację musi być aktywny
- nie może istnieć inna aktywna rezerwacja tej samej sali, która nakłada się na ten sam przedział czasu
- rezerwację można anulować wyłącznie wtedy, gdy jej termin jeszcze się nie rozpoczął
- użytkownik nie może posiadać innej aktywnej rezerwacji, która nakłada się na ten sam przedział czasu

Relacje między encjami:

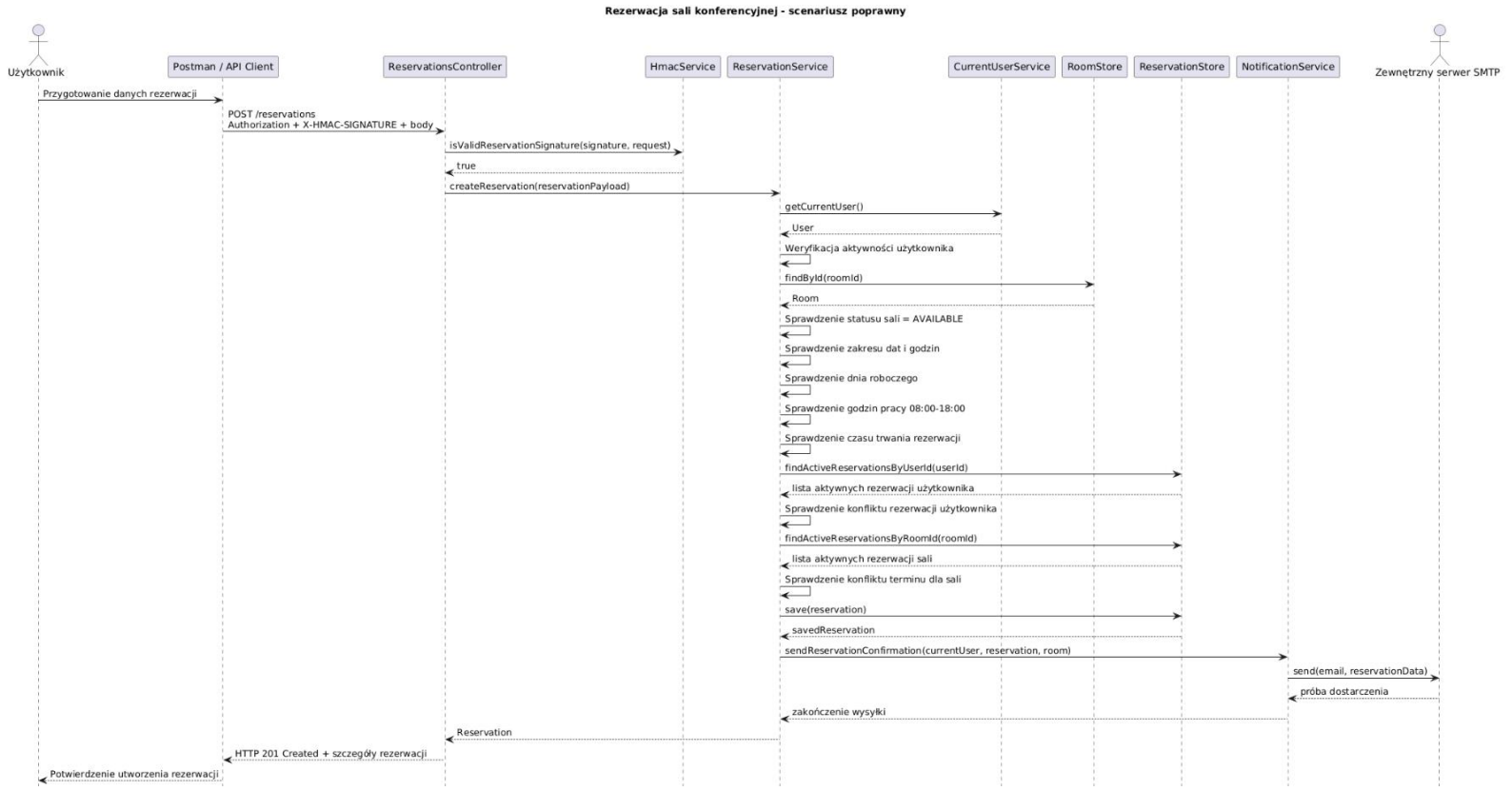
- jeden User może posiadać wiele Reservation
- jeden Room może posiadać wiele Reservation
- każde Reservation dotyczy dokładnie jednego User i jednego Room

Proces biznesowy

„Rezerwacja sali konferencyjnej”

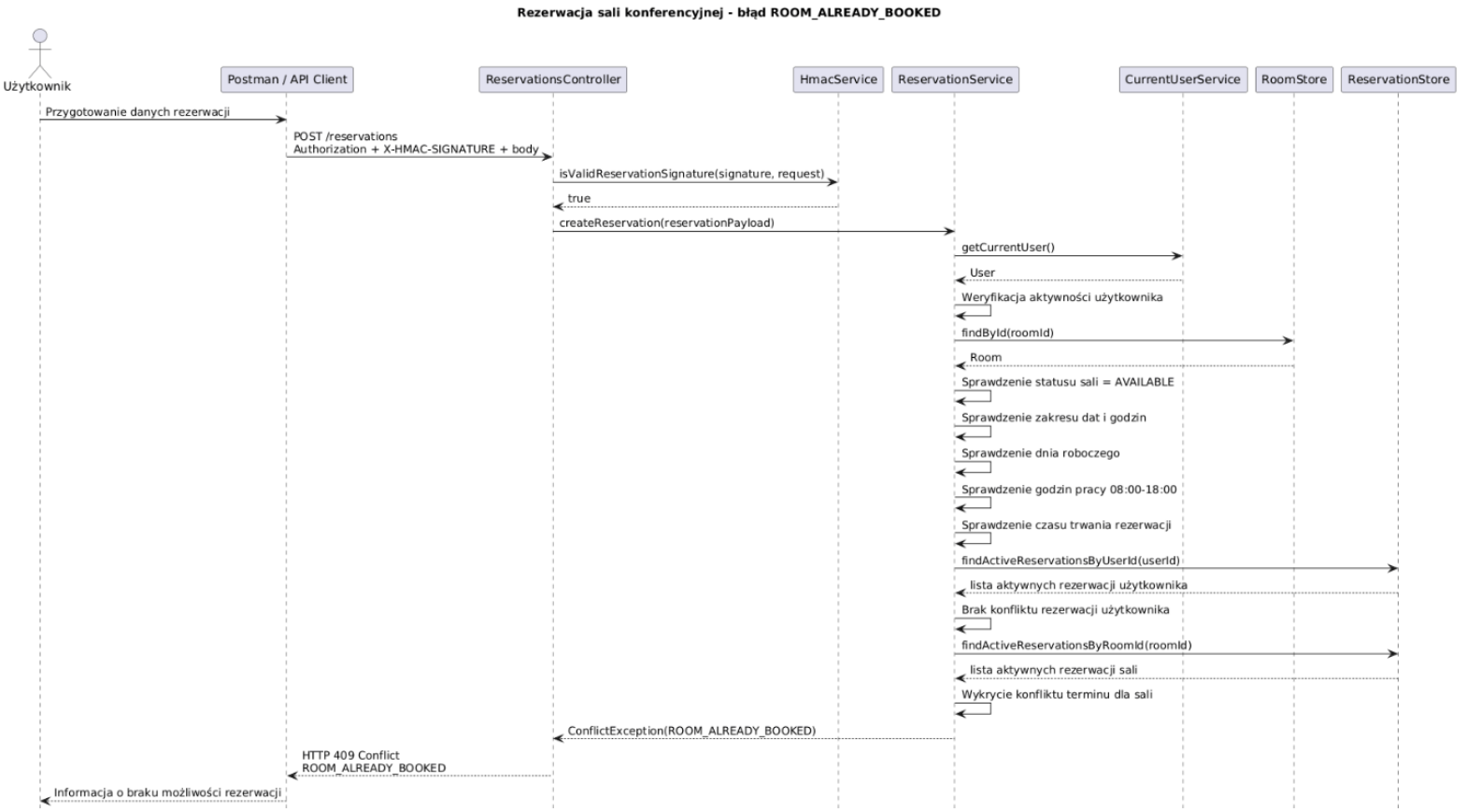
Scenariusz poprawny

1. użytkownik przygotowuje dane rezerwacji,
2. użytkownik wysyła żądanie utworzenia rezerwacji,
3. system weryfikuje poprawność podpisu HMAC przekazanego w żądaniu,
4. system pobiera dane aktualnie zalogowanego użytkownika,
5. system weryfikuje:
   - czy aktualnie zalogowany użytkownik jest aktywny,
   - czy wskazana sala istnieje,
   - czy sala ma status AVAILABLE,
   - czy zakres dat i godzin jest poprawny,
   - czy rezerwacja przypada na dzień roboczy,
   - czy rezerwacja mieści się w godzinach pracy organizacji,
   - czy czas trwania rezerwacji jest zgodny z regułami systemu,
   - czy użytkownik nie posiada już innej aktywnej rezerwacji w tym samym przedziale czasu,
   - czy nie istnieje konflikt terminu z inną aktywną rezerwacją tej samej sali,
6. jeśli wszystkie warunki są spełnione, system zapisuje rezerwację,
7. system uruchamia moduł powiadomień, który przekazuje wiadomość do zewnętrznego serwera SMTP w celu wysłania potwierdzenia na adres e-mail użytkownika,
8. system zwraca odpowiedź potwierdzającą poprawne utworzenie rezerwacji.



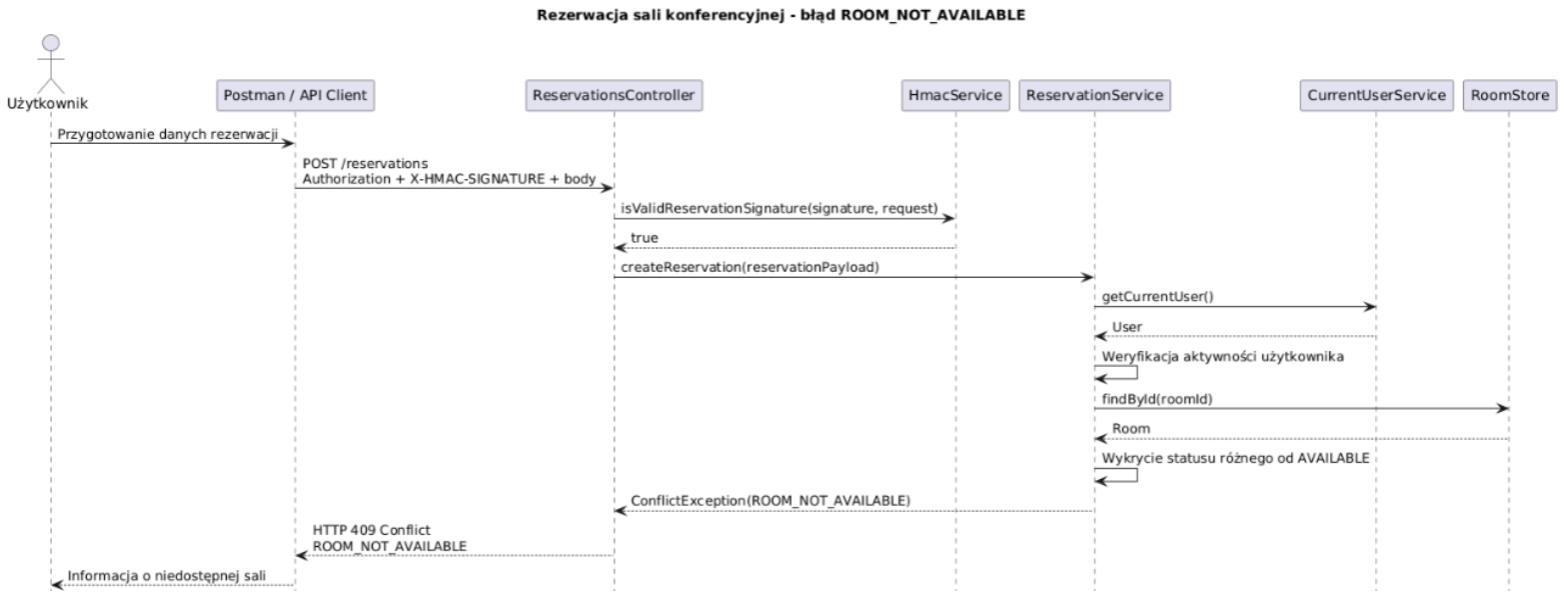
Scenariusz błędny 1 – sala jest już zarezerwowana

1. użytkownik przygotowuje dane rezerwacji,
2. użytkownik wysyła żądanie utworzenia rezerwacji,
3. system weryfikuje poprawność podpisu HMAC przekazanego w żądaniu,
4. system pobiera dane aktualnie zalogowanego użytkownika,
5. system weryfikuje:
   - czy aktualnie zalogowany użytkownik jest aktywny,
   - czy wskazana sala istnieje,
   - czy sala ma status AVAILABLE,
   - czy zakres dat i godzin jest poprawny,
   - czy rezerwacja przypada na dzień roboczy,
   - czy rezerwacja mieści się w godzinach pracy organizacji,
   - czy czas trwania rezerwacji jest zgodny z regułami systemu,
   - czy użytkownik nie posiada już innej aktywnej rezerwacji w tym samym przedziale czasu,
   - czy dla wskazanej sali nie istnieje inna aktywna rezerwacja w tym samym przedziale czasu,
6. system wykrywa konflikt terminu z istniejącą aktywną rezerwacją tej samej sali,
7. system odrzuca operację,
8. system zwraca błąd biznesowy ROOM_ALREADY_BOOKED.



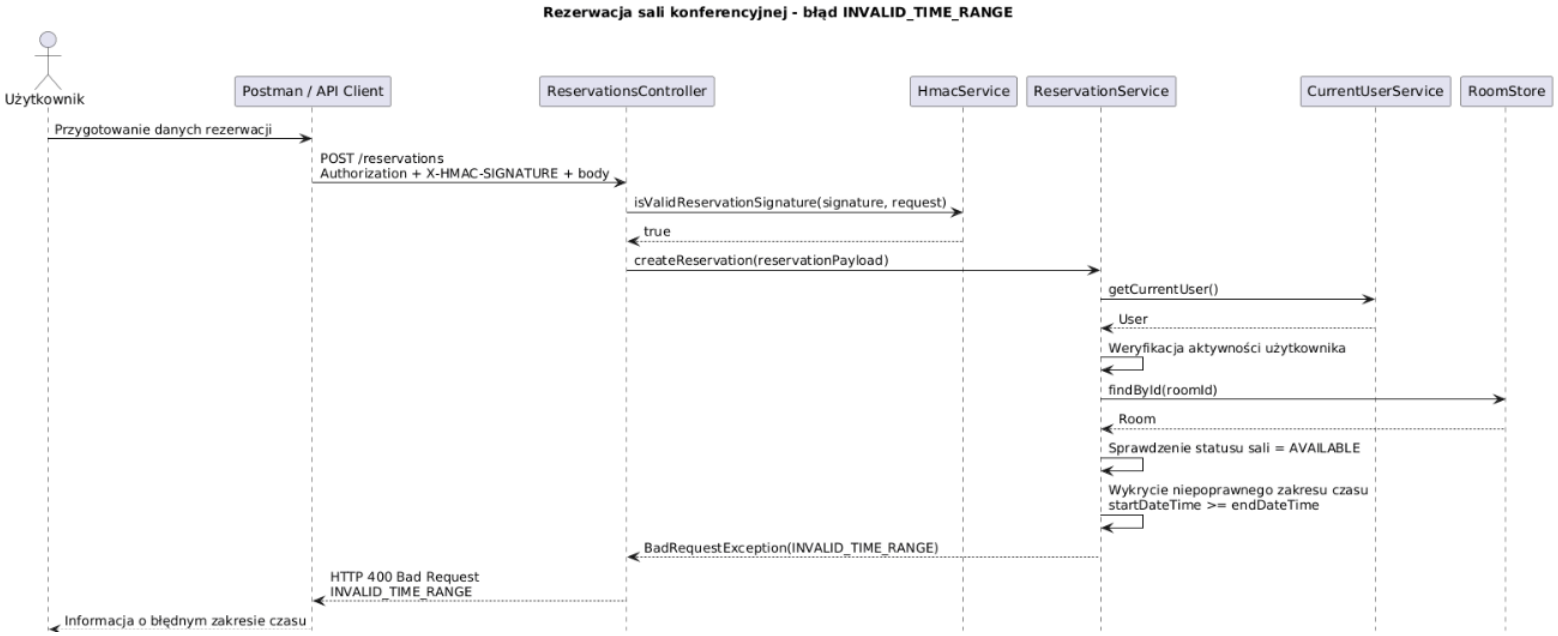
Scenariusz błędny 2 — sala jest niedostępna

1. użytkownik przygotowuje dane rezerwacji,
2. użytkownik wysyła żądanie utworzenia rezerwacji,
3. system weryfikuje poprawność podpisu HMAC przekazanego w żądaniu,
4. system pobiera dane aktualnie zalogowanego użytkownika,
5. system weryfikuje:
   - czy aktualnie zalogowany użytkownik jest aktywny,
   - czy wskazana sala istnieje,
   - czy sala ma status AVAILABLE,
6. system wykrywa, że wybrana sala nie ma statusu AVAILABLE,
7. system odrzuca operację,
8. system zwraca błąd biznesowy ROOM_NOT_AVAILABLE.



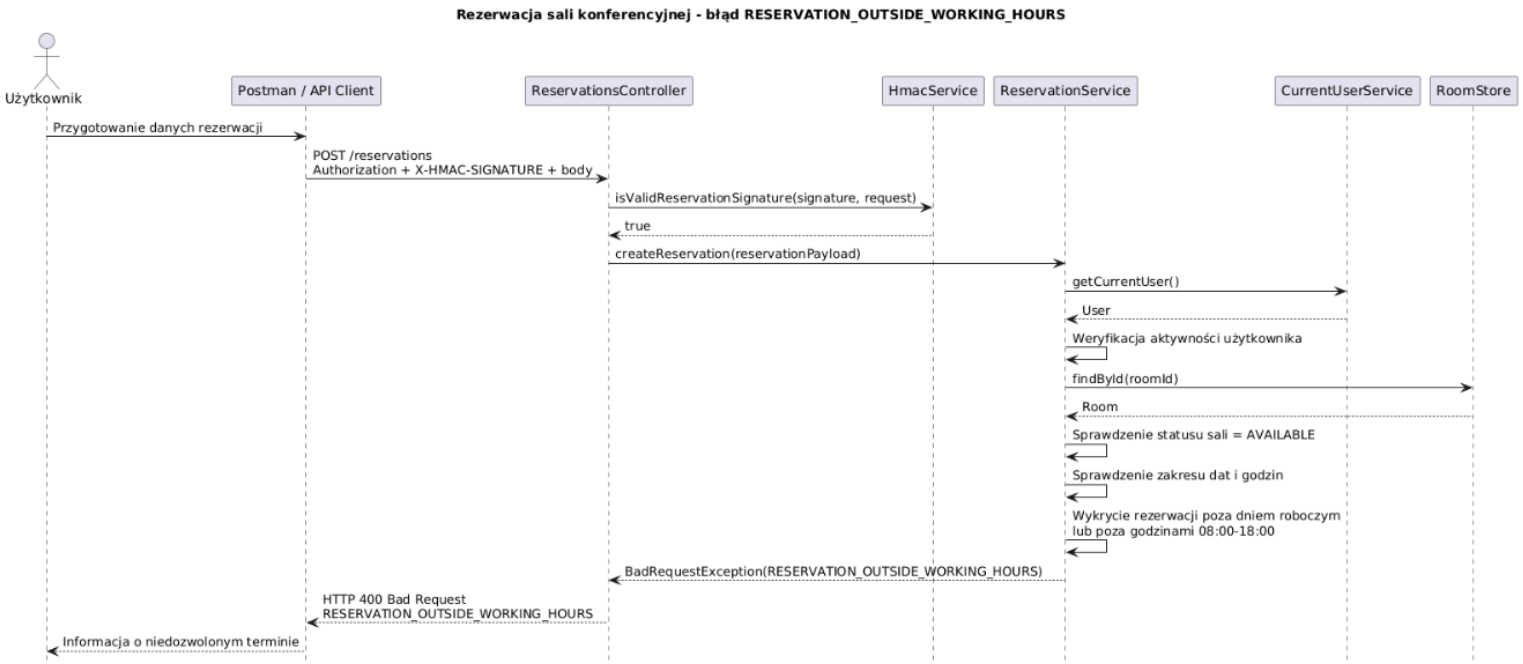
Scenariusz błędny 3 — niepoprawny zakres czasu rezerwacji

1. użytkownik przygotowuje dane rezerwacji,
2. użytkownik wysyła żądanie utworzenia rezerwacji,
3. system weryfikuje poprawność podpisu HMAC przekazanego w żądaniu,
4. system pobiera dane aktualnie zalogowanego użytkownika,
5. system weryfikuje:
   - czy aktualnie zalogowany użytkownik jest aktywny,
   - czy wskazana sala istnieje,
   - czy sala ma status AVAILABLE,
   - czy zakres dat i godzin jest poprawny,
6. system wykrywa, że moment rozpoczęcia rezerwacji nie jest wcześniejszy niż moment jej zakończenia,
7. system odrzuca operację,
8. system zwraca błąd biznesowy INVALID_TIME_RANGE.



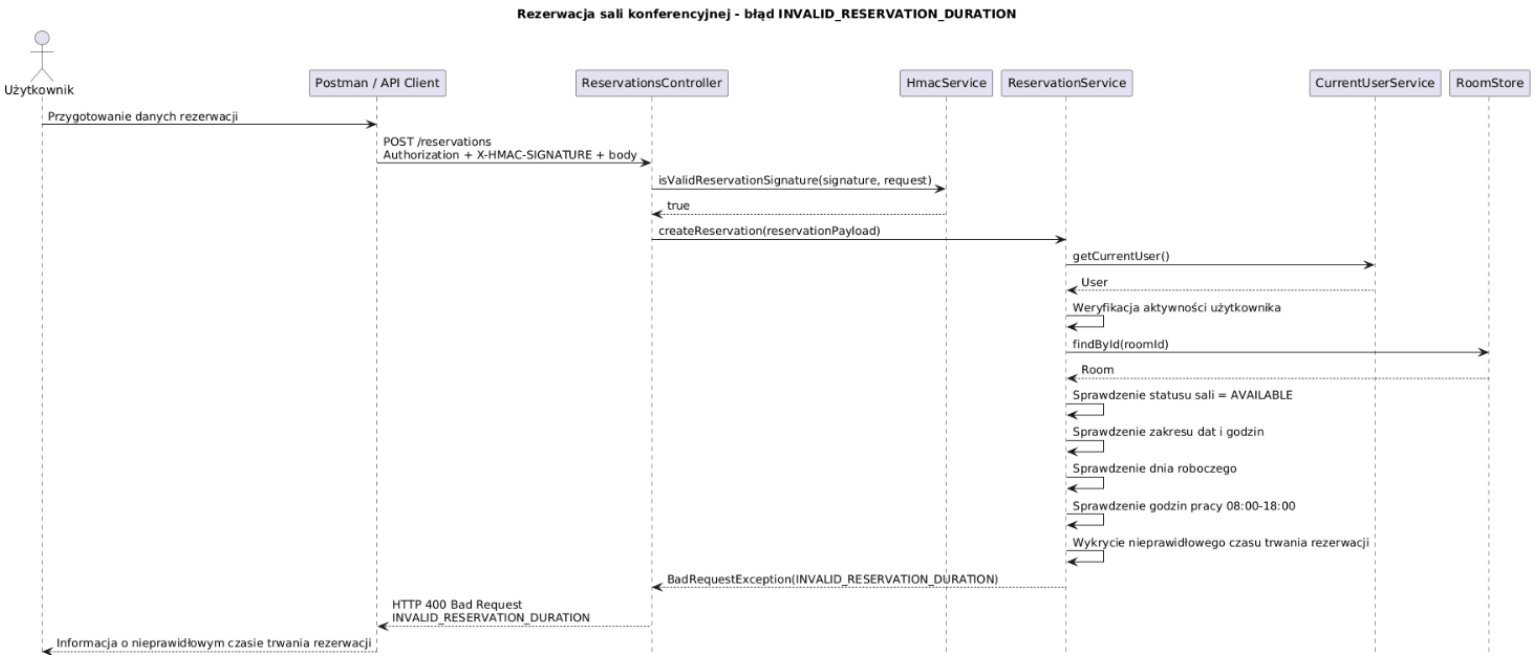
Scenariusz błędny 4 — rezerwacja poza godzinami pracy lub poza dniem roboczym

1. użytkownik przygotowuje dane rezerwacji,
2. użytkownik wysyła żądanie utworzenia rezerwacji,
3. system weryfikuje poprawność podpisu HMAC przekazanego w żądaniu,
4. system pobiera dane aktualnie zalogowanego użytkownika,
5. system weryfikuje:
   - czy aktualnie zalogowany użytkownik jest aktywny,
   - czy wskazana sala istnieje,
   - czy sala ma status AVAILABLE,
   - czy zakres dat i godzin jest poprawny,
   - czy rezerwacja przypada na dzień roboczy,
   - czy rezerwacja mieści się w godzinach pracy organizacji,
6. system wykrywa, że rezerwacja została zaplanowana poza dniami roboczymi lub poza godzinami pracy organizacji,
7. system odrzuca operację,
8. system zwraca błąd biznesowy RESERVATION_OUTSIDE_WORKING_HOURS.



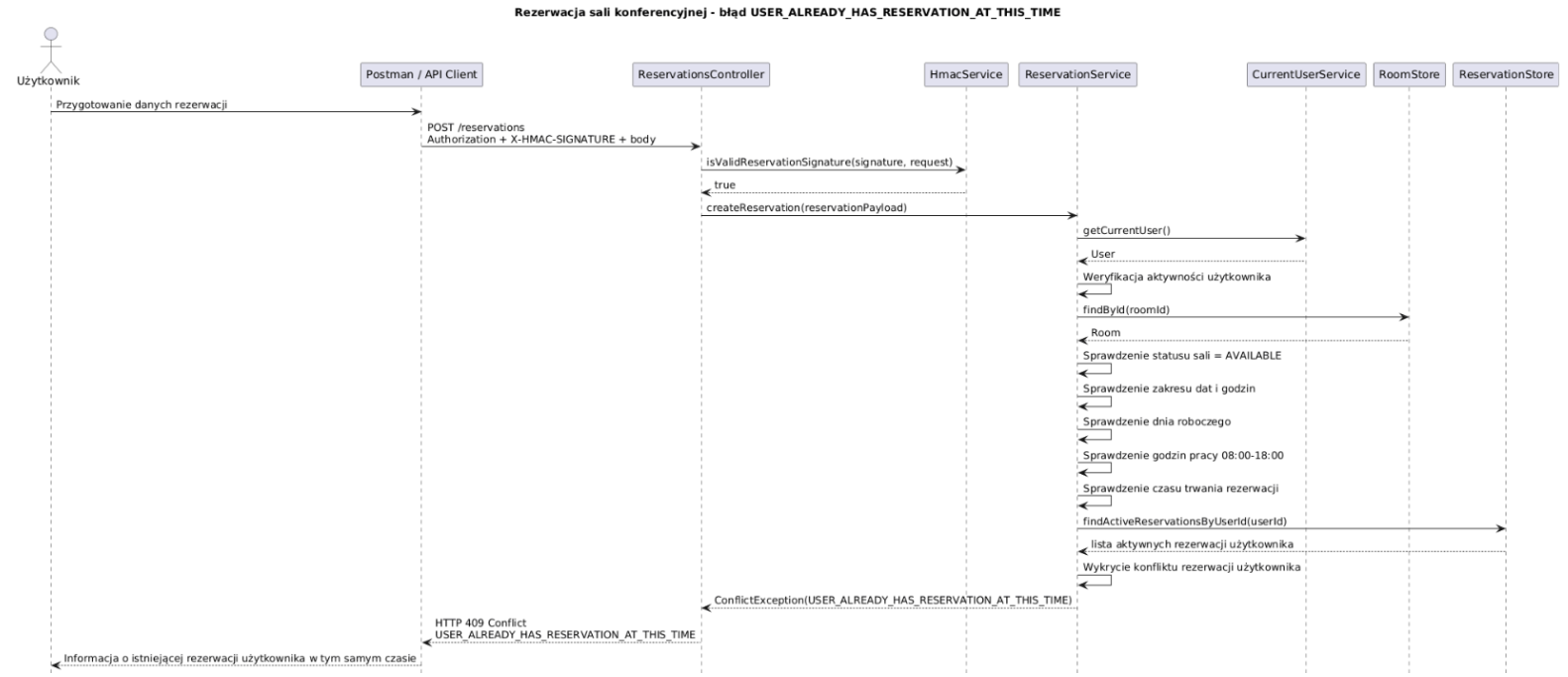
Scenariusz błędny 5 — nieprawidłowy czas trwania rezerwacji

1. użytkownik przygotowuje dane rezerwacji,
2. użytkownik wysyła żądanie utworzenia rezerwacji,
3. system weryfikuje poprawność podpisu HMAC przekazanego w żądaniu,
4. system pobiera dane aktualnie zalogowanego użytkownika,
5. system weryfikuje:
   - czy aktualnie zalogowany użytkownik jest aktywny,
   - czy wskazana sala istnieje,
   - czy sala ma status AVAILABLE,
   - czy zakres dat i godzin jest poprawny,
   - czy rezerwacja przypada na dzień roboczy,
   - czy rezerwacja mieści się w godzinach pracy organizacji,
   - czy czas trwania rezerwacji jest zgodny z regułami systemu,
6. system wykrywa, że czas trwania rezerwacji jest zbyt krótki, zbyt długi albo niezgodny z krokiem 30 minut,
7. system odrzuca operację,
8. system zwraca błąd biznesowy INVALID_RESERVATION_DURATION.



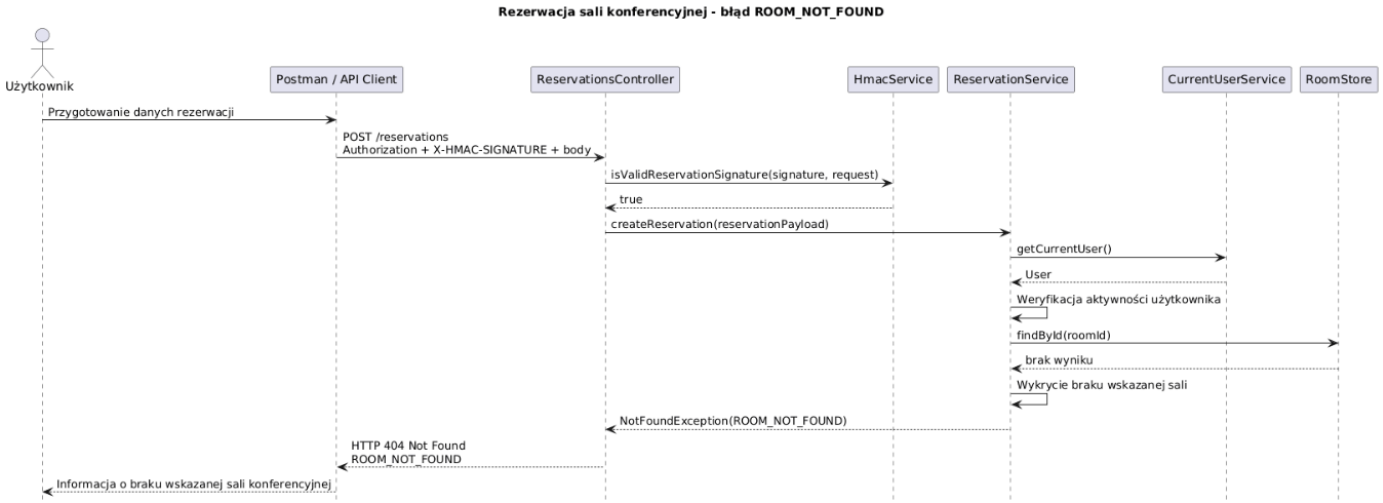
Scenariusz błędny 6 — użytkownik ma już inną rezerwację w tym samym czasie

1. użytkownik przygotowuje dane rezerwacji,
2. użytkownik wysyła żądanie utworzenia rezerwacji,
3. system weryfikuje poprawność podpisu HMAC przekazanego w żądaniu,
4. system pobiera dane aktualnie zalogowanego użytkownika,
5. system weryfikuje:
   - czy aktualnie zalogowany użytkownik jest aktywny,
   - czy wskazana sala istnieje,
   - czy sala ma status AVAILABLE,
   - czy zakres dat i godzin jest poprawny,
   - czy rezerwacja przypada na dzień roboczy,
   - czy rezerwacja mieści się w godzinach pracy organizacji,
   - czy czas trwania rezerwacji jest zgodny z regułami systemu,
   - czy użytkownik nie posiada już innej aktywnej rezerwacji w tym samym przedziale czasu,
6. system wykrywa, że użytkownik ma już inną aktywną rezerwację nakładającą się na wybrany termin,
7. system odrzuca operację,
8. system zwraca błąd biznesowy USER_ALREADY_HAS_RESERVATION_AT_THIS_TIME.



Scenariusz błędny 7 — wskazana sala konferencyjna nie została znaleziona

1. użytkownik przygotowuje dane rezerwacji,
2. użytkownik wysyła żądanie utworzenia rezerwacji,
3. system weryfikuje poprawność podpisu HMAC przekazanego w żądaniu,
4. system pobiera dane aktualnie zalogowanego użytkownika,
5. system weryfikuje:
   - czy aktualnie zalogowany użytkownik jest aktywny,
   - czy wskazana sala istnieje,
6. system wykrywa, że sala o podanym identyfikatorze nie istnieje w systemie,
7. system odrzuca operację,
8. system zwraca błąd biznesowy ROOM_NOT_FOUND.



Scenariusz błędny 8 — brak uprawnień do wykonania operacji

1. użytkownik przygotowuje dane rezerwacji,
2. użytkownik wysyła żądanie utworzenia rezerwacji,
3. system weryfikuje poprawność podpisu HMAC przekazanego w nagłówku żądania,
4. system wykrywa, że podpis HMAC jest niepoprawny lub niezgodny z treścią żądania,
5. system odrzuca operację,
6. system zwraca błąd FORBIDDEN.



Zastosowane mechanizmy bezpieczeństwa w aplikacji:

W opracowanej aplikacji zastosowano kilka mechanizmów bezpieczeństwa, których celem jest ochrona zasobów systemu, integralności żądań oraz poprawności przetwarzanych danych.

1. Uwierzytelnianie użytkowników

Aplikacja wykorzystuje mechanizm HTTP Basic Authentication, dzięki któremu dostęp do endpointów API jest możliwy wyłącznie po podaniu poprawnych danych logowania użytkownika. Każde żądanie do chronionych zasobów wymaga przesłania nagłówka Authorization.

2. Autoryzacja oparta na rolach

W systemie zaimplementowano autoryzację opartą na rolach użytkowników. Wyróżniono dwie role:

- USER
- ADMIN

Dostęp do wybranych operacji został ograniczony zgodnie z uprawnieniami danej roli. Przykładowo administrator może zarządzać salami konferencyjnymi oraz przeglądać wszystkie rezerwacje, natomiast zwykły użytkownik ma dostęp jedynie do operacji dotyczących własnych danych i własnych rezerwacji.

3. Weryfikacja integralności żądań z użyciem HMAC

Dla operacji tworzenia rezerwacji (POST /reservations) zastosowano dodatkowy mechanizm ochrony integralności danych w postaci podpisu **HMAC** przekazywanego w nagłówku X-HMAC-SIGNATURE.
Podpis ten jest wyliczany na podstawie treści żądania i współdzielonego sekretu, a następnie porównywany po stronie serwera. Rozwiązanie to zabezpiecza system przed nieautoryzowaną modyfikacją danych żądania.

4. Walidacja danych wejściowych

W aplikacji zastosowano zarówno walidację syntaktyczną, jak i walidację biznesową danych wejściowych. Obejmuje ona między innymi:

- sprawdzanie wymaganych pól,
- poprawność zakresu dat i godzin,
- kontrolę czasu trwania rezerwacji,
- weryfikację dostępności sali,
- wykrywanie konfliktów rezerwacji użytkownika i sali,
- sprawdzanie zgodności rezerwacji z godzinami pracy organizacji.

Dzięki temu system odrzuca dane niepoprawne jeszcze przed wykonaniem operacji biznesowej.

5. Kontrolowana obsługa błędów

Aplikacja wykorzystuje własne klasy wyjątków oraz ujednolicony model odpowiedzi błędów. W przypadku problemów system zwraca kontrolowane komunikaty wraz z odpowiednimi kodami HTTP, np. 400, 401, 403, 404 czy 409.
Takie rozwiązanie zwiększa przewidywalność działania API i ogranicza ujawnianie wewnętrznych szczegółów implementacyjnych.

6. Ochrona danych wrażliwych i sekretów konfiguracyjnych

Dane wrażliwe, takie jak sekrety konfiguracyjne czy dane do wysyłki poczty, zostały wydzielone do osobnego pliku konfiguracyjnego, który nie jest publikowany w repozytorium Git.
Pozwala to ograniczyć ryzyko ujawnienia poufnych danych w publicznie dostępnych źródłach projektu.

7. Bezpieczna obsługa haseł

W aplikacji zastosowano mechanizm PasswordEncoder, dzięki któremu proces uwierzytelniania nie opiera się na porównywaniu haseł w postaci jawnej. Rozwiązanie to zwiększa bezpieczeństwo obsługi kont użytkowników.

Uzasadnienie dla mechanizmów bezpieczeństwa

1. Uwierzytelnianie — Basic Auth

Uzasadnienie wyboru:
Mechanizm Basic Auth został wybrany ze względu na prostotę implementacji oraz wystarczający poziom funkcjonalności dla aplikacji projektowej o charakterze demonstracyjnym. Rozwiązanie to dobrze integruje się z frameworkiem Spring Security i pozwala w prosty sposób zabezpieczyć dostęp do endpointów API. W warunkach produkcyjnych mechanizm ten powinien być stosowany wyłącznie z użyciem szyfrowanego połączenia HTTPS.

2. Autoryzacja oparta o role

Uzasadnienie wyboru:
System rezerwacji sal naturalnie wymaga rozróżnienia użytkowników o różnych uprawnieniach. Zastosowanie ról USER i ADMIN pozwala w przejrzysty sposób oddzielić operacje zwykłych pracowników od operacji administracyjnych. Rozwiązanie to zwiększa bezpieczeństwo systemu i ogranicza możliwość wykonania nieuprawnionych działań.

3. Integralność komunikacji — HMAC

Uzasadnienie wyboru:
Mechanizm HMAC został zastosowany do ochrony integralności żądań tworzenia rezerwacji. Dzięki temu serwer może zweryfikować, czy przesłane dane nie zostały zmodyfikowane pomiędzy klientem a aplikacją. Rozwiązanie to jest szczególnie istotne dla operacji zmieniających stan systemu i stanowi dodatkową warstwę zabezpieczenia oprócz standardowego uwierzytelnienia.

4. Walidacja danych wejściowych

Uzasadnienie wyboru:
Walidacja danych wejściowych została zastosowana w celu ochrony systemu przed niepoprawnymi, niespójnymi lub niezgodnymi z regułami biznesowymi danymi. Mechanizm ten zapobiega powstawaniu błędnych rezerwacji, konfliktów logicznych oraz nieprawidłowych operacji na danych.

5. Kontrolowana obsługa błędów

Uzasadnienie wyboru:
Własna obsługa wyjątków i ujednolicony model odpowiedzi błędów pozwalają zachować przewidywalność działania API, ułatwiają diagnostykę problemów po stronie klienta oraz ograniczają ujawnianie wewnętrznych szczegółów implementacyjnych. Rozwiązanie to poprawia zarówno bezpieczeństwo, jak i czytelność interfejsu API.

6. Wydzielenie sekretów do osobnego pliku konfiguracyjnego

Uzasadnienie wyboru:
Przechowywanie sekretów poza głównym kodem aplikacji zmniejsza ryzyko ich przypadkowego opublikowania w repozytorium. Jest to dobre podejście organizacyjne i bezpieczeństwa, szczególnie przy pracy z danymi konfiguracyjnymi związanymi z pocztą lub podpisami HMAC.

7. Bezpieczna obsługa haseł

Uzasadnienie wyboru:
Zastosowanie PasswordEncoder ogranicza ryzyko związane z przechowywaniem i porównywaniem haseł w postaci jawnej. Jest to standardowe i zalecane rozwiązanie w aplikacjach korzystających z mechanizmów uwierzytelniania użytkowników.

Błędy biznesowe:

ROOM_ALREADY_BOOKED

Sala jest już zarezerwowana w wybranym terminie.

ROOM_NOT_AVAILABLE

Sala nie może zostać zarezerwowana, ponieważ nie ma statusu AVAILABLE.

INVALID_TIME_RANGE

Zakres dat i godzin rezerwacji jest niepoprawny, ponieważ moment rozpoczęcia nie jest wcześniejszy niż moment zakończenia.

RESERVATION_OUTSIDE_WORKING_HOURS

Rezerwacja została zaplanowana poza dniami roboczymi lub poza godzinami pracy organizacji, tj. poza przedziałem 08:00–18:00.

INVALID_RESERVATION_DURATION

Czas trwania rezerwacji jest nieprawidłowy, ponieważ jest zbyt krótki, zbyt długi albo niezgodny z krokiem 30 minut.

USER_ALREADY_HAS_RESERVATION_AT_THIS_TIME

Użytkownik posiada już inną aktywną rezerwację w tym samym przedziale czasu.

RESERVATION_CANNOT_BE_CANCELLED

Nie można anulować rezerwacji, która już się rozpoczęła lub już się zakończyła.

ROOM_NOT_FOUND

Wskazana sala konferencyjna nie została znaleziona.

INVALID_REQUEST

Niepoprawne dane wejściowe.

FORBIDDEN (niepoprawny HMAC)

Użytkownik nie posiada uprawnień do wykonania tej operacji.

Endpointy:

Użytkownik

GET /users/me
Zwraca dane aktualnie zalogowanego użytkownika.

GET /users/me/reservations
Zwraca listę rezerwacji aktualnie zalogowanego użytkownika.

Sale konferencyjne

GET /rooms
Zwraca listę wszystkich sal konferencyjnych.

GET /rooms/{id}
Zwraca szczegóły wybranej sali konferencyjnej.

POST /rooms
Dodaje nową salę konferencyjną. Dostęp: ADMIN

PUT /rooms/{id}
Edytuje dane sali konferencyjnej. Dostęp: ADMIN

PATCH /rooms/{id}/status
Zmienia status sali konferencyjnej. Dostęp: ADMIN

Rezerwacje

POST /reservations
Tworzy nową rezerwację sali konferencyjnej. Dostęp: USER, ADMIN. Dodatkowo: HMAC

GET /reservations/{id}
Zwraca szczegóły jednej rezerwacji. USER może przeglądać własne rezerwacje, ADMIN wszystkie.

GET /reservations
Zwraca listę wszystkich rezerwacji. Dostęp: ADMIN

POST /reservations/{id}/cancel
Anuluje rezerwację. USER może anulować własną rezerwację, ADMIN może anulować dowolną.

Integracja z usługą zewnętrzną:

W projekcie zaimplementowano integrację z zewnętrzną usługą pocztową (SMTP), umożliwiającą automatyczne wysyłanie powiadomień e-mail do użytkownika po utworzeniu rezerwacji.

